



COMPUTER SECURITY

Lecture 3: Cryptography (Cont.)

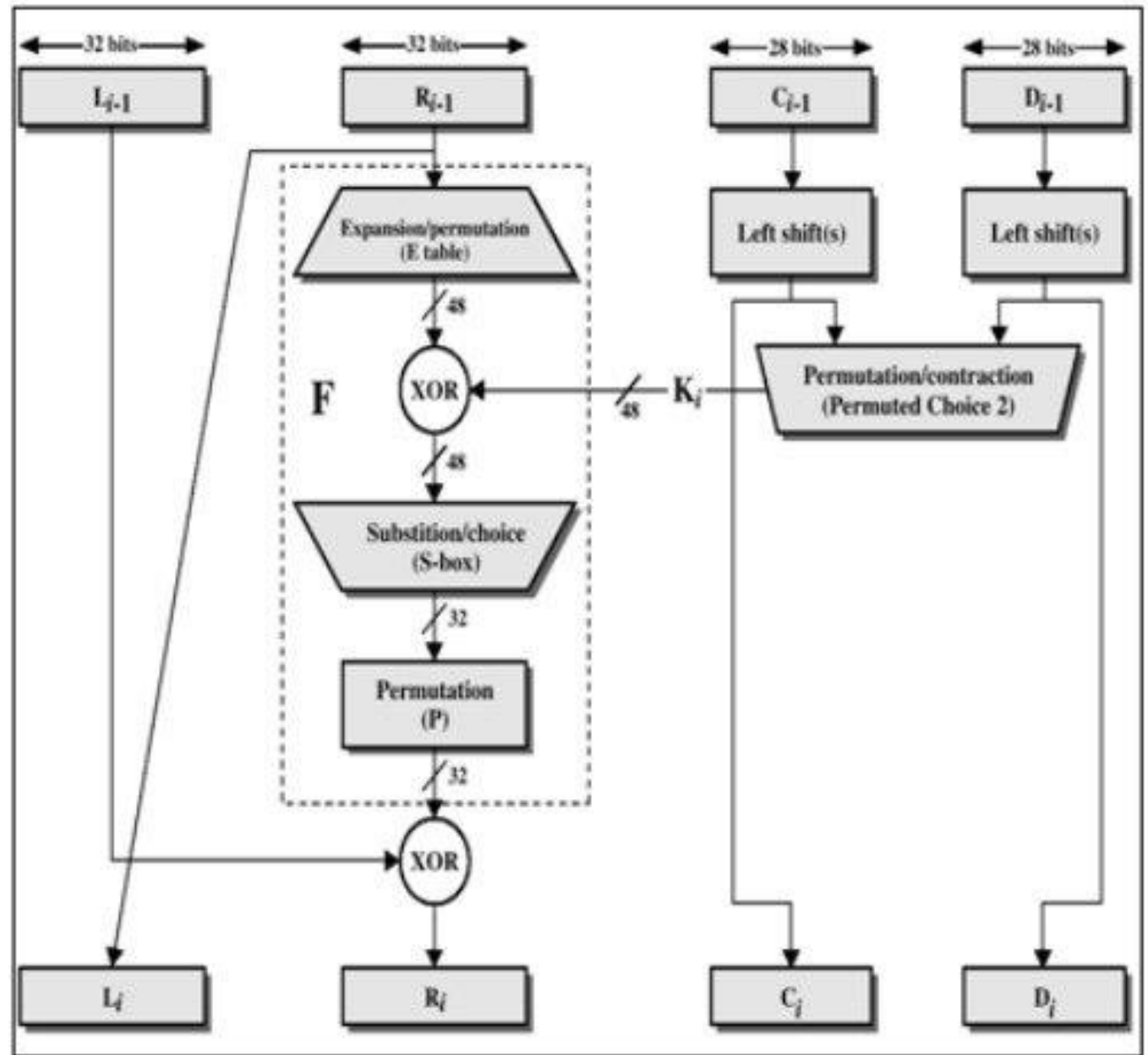
Prof. Dr. Noha Hikal

**Information Technology Dept.
Vice dean for Community Services and Environmental Development**

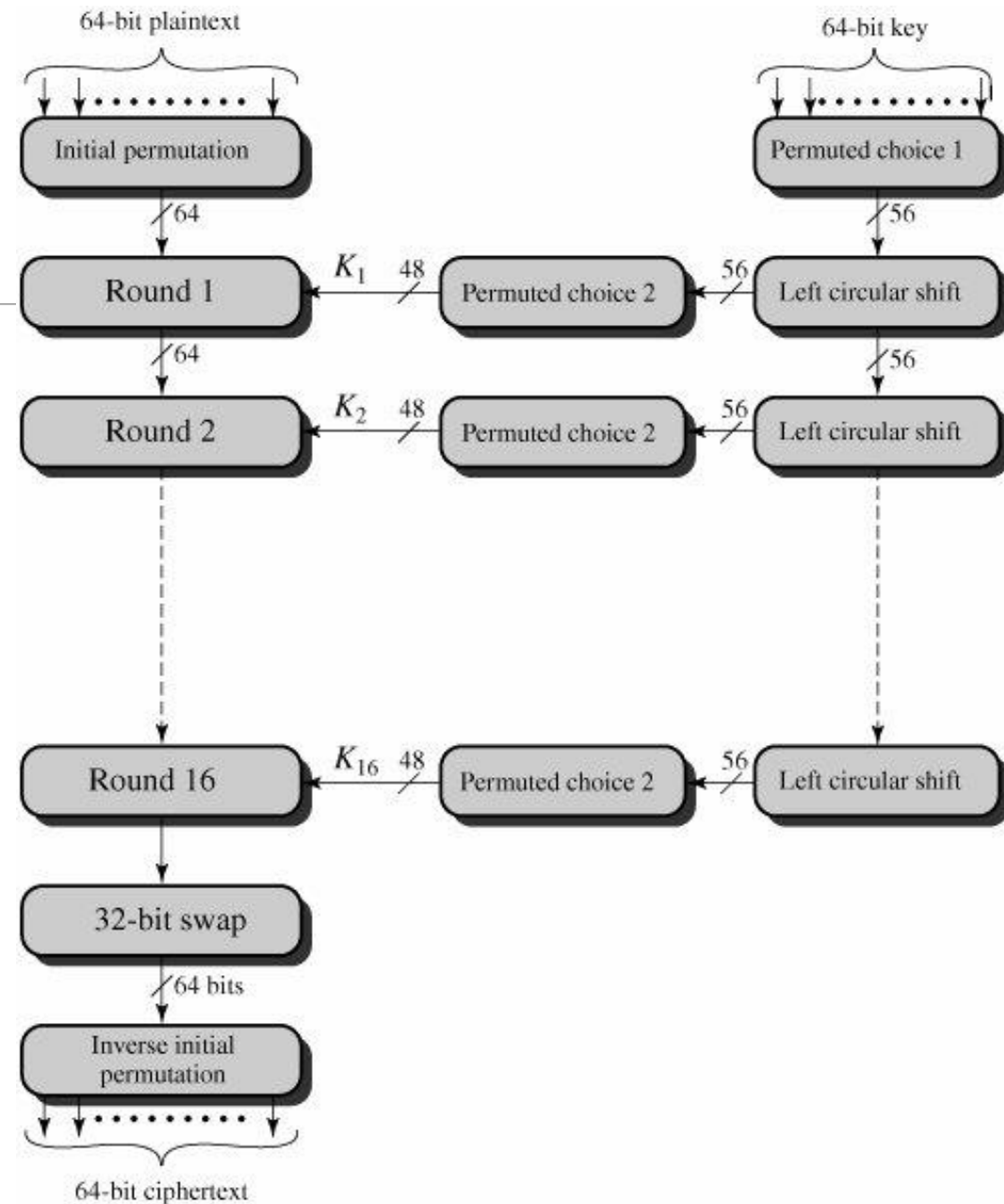
CONTENTS:

- DES round structure
- 2DES
- 3DES
- Public key cryptography
- Public-Key Applications
- Key management & distribution
- Certificates
- Key maintenance

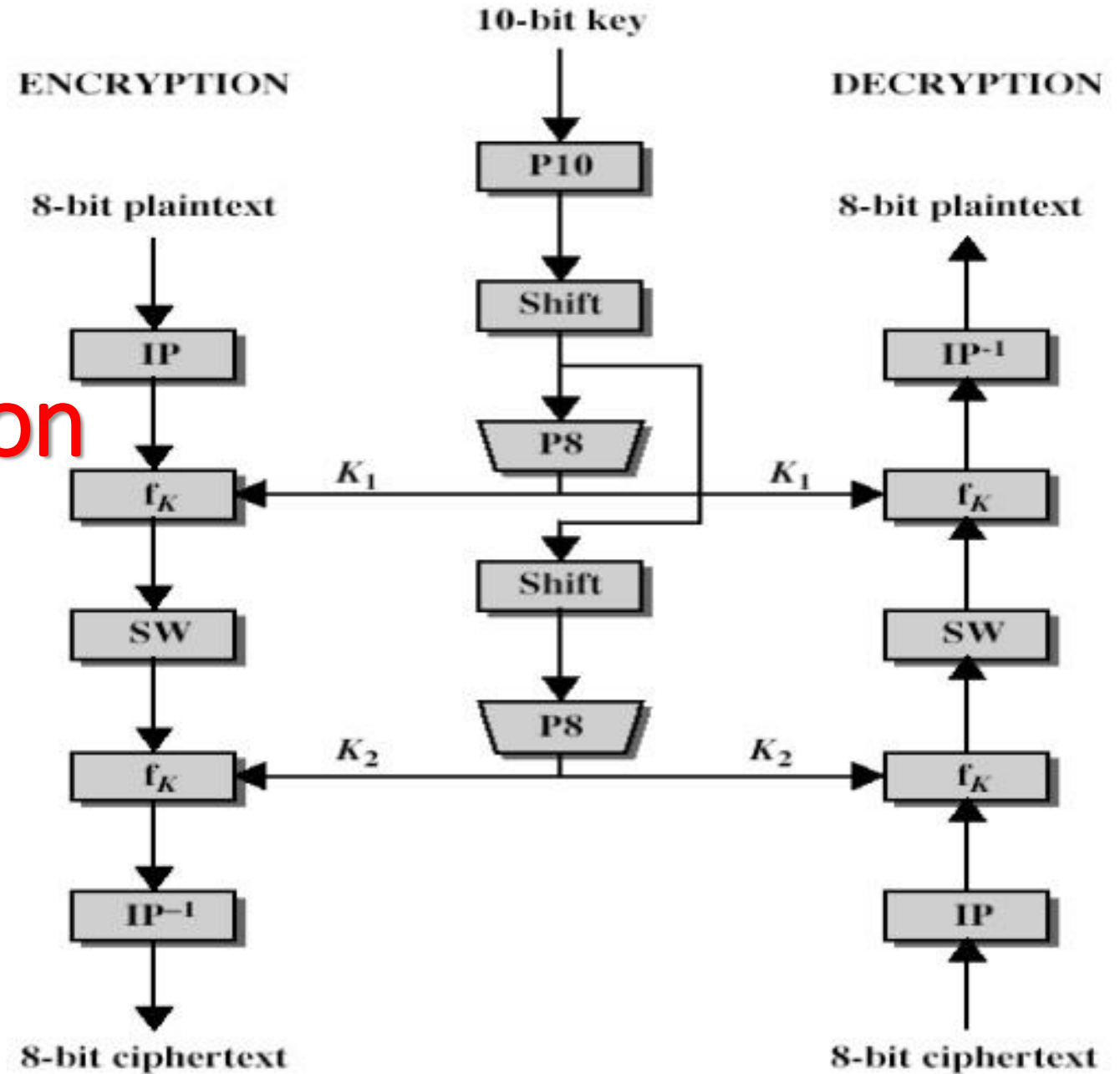
1- DES round structure



DES Structure



DES Encryption/Decryption Structure



2- Double - DES

The simplest form of multiple encryption has two encryption stages and two keys - Double-DES.

$$C = Ek_1(Ek_2(P))$$

Vulnerable to man-in-the middle attack

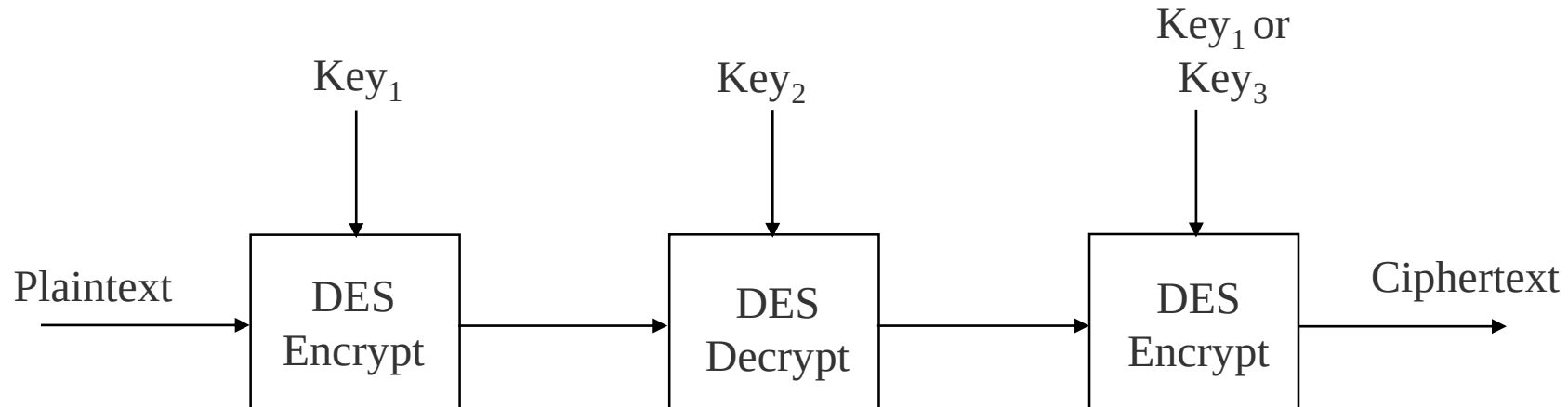
3- Triple DES

With Triple DES, each block of plaintext is processed three times

The two-key version of triple DES is estimated to be 10^{13} times stronger than single-DES. The three-key version is even stronger.

If Key_1 , Key_2 and Key_3 are all the same the Triple-Des collapses to Single DES.

The issue with Triple-Des is Key Management.



4- Public-Key Cryptography

public-key/two-key/asymmetric cryptography involves the use of **two** keys:

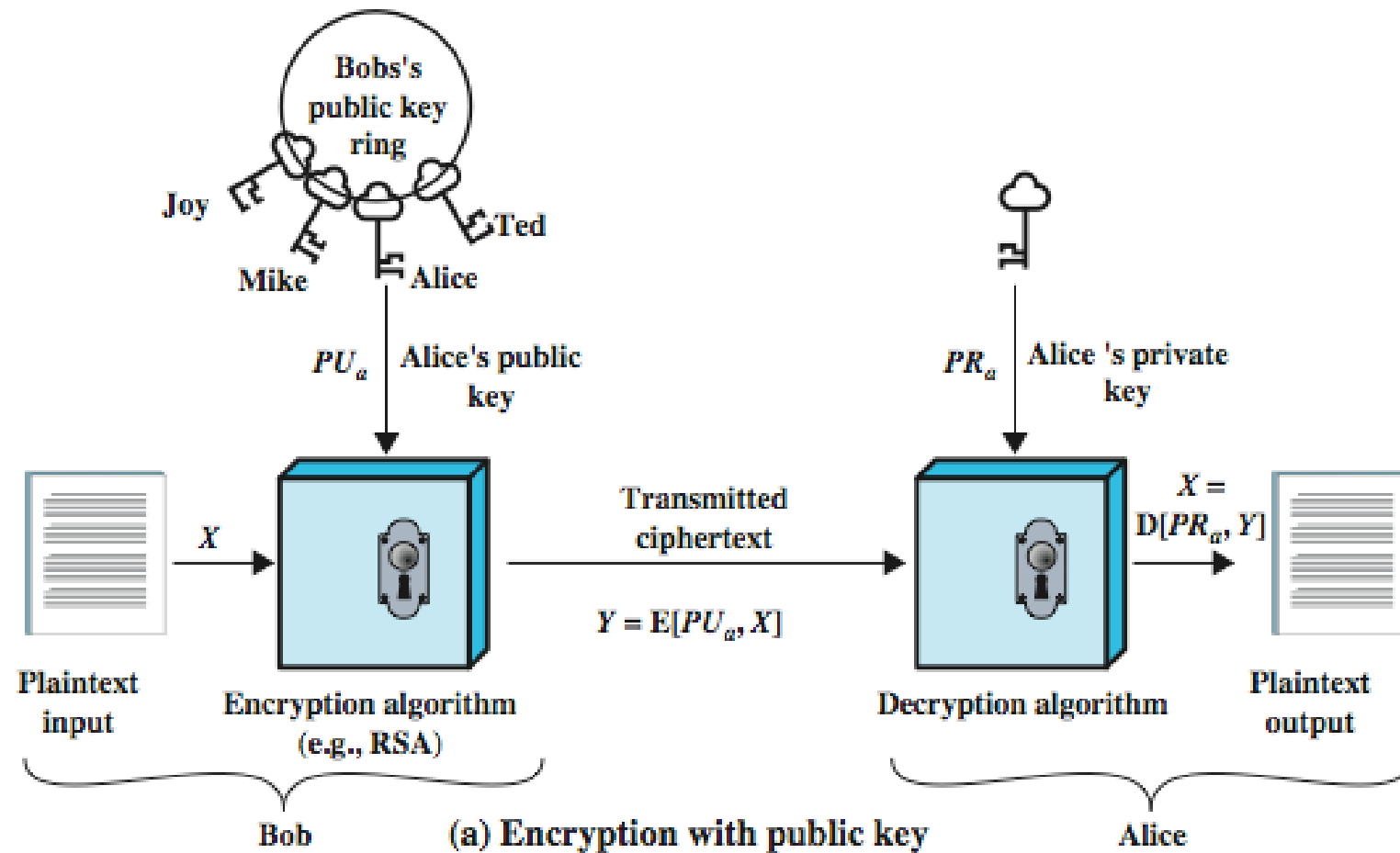
- a **public-key**, which may be known by anybody, and can be used to **encrypt messages**, and **verify signatures**
- a related **private-key**, known only to the recipient, used to **decrypt messages**, and **sign** (create) **signatures**

infeasible to determine private key from public

is **asymmetric** because

- those who encrypt messages or verify signatures **cannot** decrypt messages or create signatures

Cont.



Symmetric vs Public-Key

Conventional Encryption	Public-Key Encryption
<i>Needed to Work:</i> 1. The same algorithm with the same key is used for encryption and decryption. 2. The sender and receiver must share the algorithm and the key. <i>Needed for Security:</i> 1. The key must be kept secret. 2. It must be impossible or at least impractical to decipher a message if no other information is available. 3. Knowledge of the algorithm plus samples of ciphertext must be insufficient to determine the key.	<i>Needed to Work:</i> 1. One algorithm is used for encryption and decryption with a pair of keys, one for encryption and one for decryption. 2. The sender and receiver must each have one of the matched pair of keys (not the same one). <i>Needed for Security:</i> 1. One of the two keys must be kept secret. 2. It must be impossible or at least impractical to decipher a message if no other information is available. 3. Knowledge of the algorithm plus one of the keys plus samples of ciphertext must be insufficient to determine the other key.

5- Public-Key Applications

can classify uses into 3 categories:

- **encryption/decryption** (provide secrecy)
- **digital signatures** (provide authentication)
- **key exchange** (of session keys)

some algorithms are suitable for all uses, others are specific to one

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

Public-Key Requirements

Public-Key algorithms rely on two keys where:

- it is computationally infeasible to find decryption key knowing only algorithm & encryption key
- it is computationally easy to en/decrypt messages when the relevant (en/decrypt) key is known
- either of the two related keys can be used for encryption, with the other used for decryption (for some algorithms)

these are formidable requirements which only a few algorithms have satisfied

Security of Public Key Schemes

- like private key schemes brute force **exhaustive search** attack is always theoretically possible
- but keys used are too large (>512bits)
- security relies on a **large enough** difference in difficulty between **easy** (en/decrypt) and **hard** (cryptanalyse) problems
- more generally the **hard** problem is known, but is made hard enough to be impractical to break
- requires the use of **very large numbers**
- hence is **slow** compared to private key schemes

Timing Attacks

developed by Paul Kocher in mid-1990's

exploit timing variations in operations

- eg. multiplying by small vs large number
- or varying which instructions executed

infer operand size based on time taken

RSA exploits time taken in exponentiation

countermeasures

- use constant exponentiation time
- add random delays
- blind values used in calculations

6- Key management and distribution

For all encryption algorithms those are a major issue

- key management. It involves storing, safeguarding, and activating keys.
- key distribution refers to the procedures by which keys are securely provided to parties legitimately asking for them.

key management security main goal :

is the control of keying material through the entire lifetime of the keys in order to prevent unauthorized disclosure, modification, substitution, replay and improper use. As follows:

- **Data Confidentiality**: secret keys and possibly other data are to be kept confidential while transmitted or stored.
- **Modification Detection**: is to counter the active threat of unauthorized modification of data items.
- **Replay Detection/Timeliness**: Reply detection is to counter unauthorized duplication of data items. Timeliness is required that the response to a challenge message is prompt and does not allow for playback of some authentic response message by an impersonator.

- **Entity Authentication** is to corroborate that an entity is the one claimed.

- **Data Origin Authentication** (*Proof/Norepudiation of Origin*) is to corroborate that the source of a message is the one claimed.
- **Proof/Norepudiation of Reception** shows the sender of a message that the message has been received by its legitimate receiver correctly.
- **Notarization** is the registration of messages to attest at a later stage its content origin

Services to ensure robust Key Management

1.Entity Registration

- Mainly used to provide some link between an entity and its uniquely defined keys.
- In any system, an entity is represented by some public data called it's public credentials such as a User-ID or an Address, and some private credentials such as testimonials or passwords.
- When an entity is registered, a certificate based upon its credentials is issued as a proof of registration.

2.Key Generation

- This refers to the procedure by which keys or pairs of keys of good cryptographic quality are securely and unpredictably generated.
- This requires the use of a good method for generating random numbers.

Services to ensure robust Key Management (cont.)

Certification (On-Line/Off-Line)

- Certificates are issued for authentication purposes. A credential containing identifying data together with other information (e.g. public keys) is rendered unforgeable by some certifying information (e.g. digital signatures provided by the key certification centre). Certification may be:
 - an on-line service where some certification authority provides interactive support and is actively involved in key distribution processes; or
 - it may be an off-line service so that certificates are issued to each entity only at some initial stage.

Authentication/Verification

- The three main types of authentication are *1) entity authentication, 2) message content authentication, and 3) message origin authentication*. The term verification refers to checking the appropriate claims, i.e. the correct identity of an entity. The validity of a certificate may can be verified using some public information (e.g. a public key), and can be carried out without the need for assistance of the certification authority.

7- Certificates

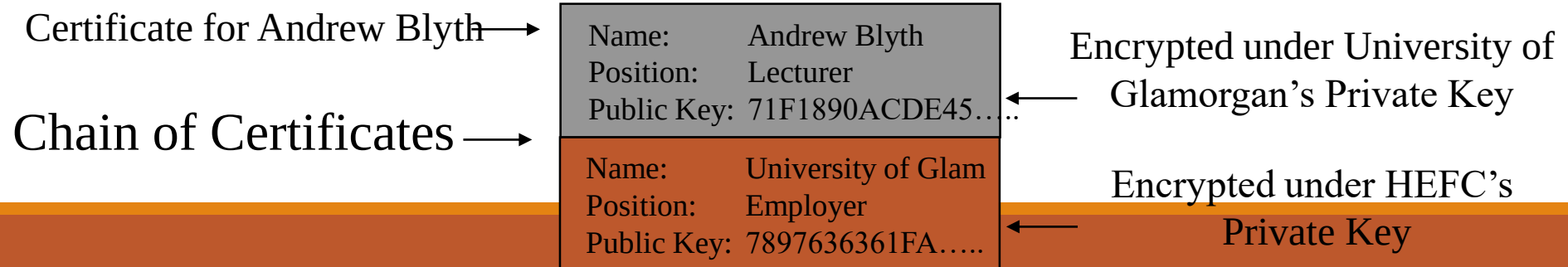
As humans we establish and use trust all the time. However trust is based upon the ability of people to identify and certify who and what they are.

- We use certificates and trust all the time to do business.

Within modern encryption systems a public key and a user's identity are bound into a certificate which is then signed by someone to certify the accuracy of that binding.

Certificates can be awarded by certification authorities (CA), and certificate authorities can themselves have certificates

Encryption keys are used to create and manage certificates, as keys may have a limited life to them.



8- Key Maintenance

- ***Storage of Keying Material*** refers to a key storage facility which provides secure storage of keys for future use, e.g. confidentiality and integrity for secret keying material, or integrity for public keys. For all keying materials unauthorized modification must be detectable by suitable authentication mechanisms.
- ***Key Archival*** refers to procedures by which keys for notarization or nonrepudiation services can be securely archived. Archived keys may need to be retrieved at a much later date.
- ***Key Replacement*** enables parties to update securely their keying material. A replaced key shall not be reused. A key shall be replaced when its compromise is known or suspected.
- ***Key Recovery*** refers to cryptographic keys which may become lost due to human error, software bugs, or hardware malfunction.
- ***Key Deletion*** refers to procedures by which parties are assured of the secure destruction of keys that are no longer needed.

Key Centres

The purpose of a Key Distribution Centre (KDC) is to generate or acquire and distribute keys to parties, each of which already shares a key with the KDC. For example they know the public-key of the KDC.

The main purpose of the Key Certification Centre (KCC) is to maintain a database of all users of a system or a certificate.

- This allows people to certify if an entity holds, or is entitled to hold a certificate.
- The KCC needs to maintain some sort of blacklist.
- Depending on the set of users it may be necessary to create a network of KCCs

1- Symmetric Key Exchange With a Server

Suppose that Pablo and Robert want a secret encryption key by which they can exchange messages, but they have no key in common. Suppose that a key repository exists such that Robert shares a key (K_R) with it and Pablo shares a key (K_P) with it.

1. First Pablo sends a message to the server containing $[P, R, I_P]$. P is the identity of Pablo, R is Pablo's intended recipient. I_P is a unique identifier
2. The key repository generates fresh encryption keys for Pablo and Robert to use.
3. The key repository sends Pablo $E(K_P, [I_P, R, K_{PR}, E(K_R, [K_{PR}, P])])$. Pablo decrypts the message and get the following:
 - The message Identify I_P
 - Robert's identification R
 - A Key for Communication, K_{PR}
 - A string containing (K_{PR}, P) that only Robert and decrypt. Note that P is Pablo's identification.
4. Pablo then send the string containing (K_{PR}, P) to Robert, and Robert decrypts it.

Kp: Pablo Key

Kr: Robert Key



1- [P, R, Ip]

2- $E(K_P, [I_P, R, K_{PR}, E(K_R, [K_{PR}, P])])$

3- (K_{PR}, P)



Pablo



Robert

2- Symmetric Key Exchange Without a Server

Assume that the two users already have a copy of a single symmetric (secret) key K known only to them. However they do not want to use K all the time, as the more data that is encrypted with K the greater is the chance that it will be compromised.

Pablo creates a new key called K_{NEW} and encrypts its using K .

- Thus $C = E(K_{NEW}, K)$. Pablo then sends the message C to Robert.

Robert decrypts the message using K and thus derives K_{NEW} .

- Thus $K_{NEW} = D(K_{NEW}, K)$.

The new key K_{NEW} is called a session key and the single symmetric (secret) key K is called a Master Key.

The disadvantage of this approach is that two users must both share one key that is unique to them. Other pairs of users need unique keys, and in general N users need $N(N-1)/2$ keys.

K: Master key , K_{NEW} : session key



1- $C = E(K_{NEW}, K)$.



2- $K_{NEW} = D(K_{NEW}, C)$.

3- Asymmetric Key Exchange With a Server

Pablo sends a message to the distribution server saying that I would like to send a message to Robert, and Pablo does not know Robert's public key.

The server responds with $E(D_D, [E_R, R])$. Remember that D_D is the private key for the distribution server, E_R is the public key for Robert and R is Robert's identification.

Pablo now sends a message $E(E_R, [P, I_P])$ to Robert using Robert's public key. This message contains his identification P and a reference number I_P .

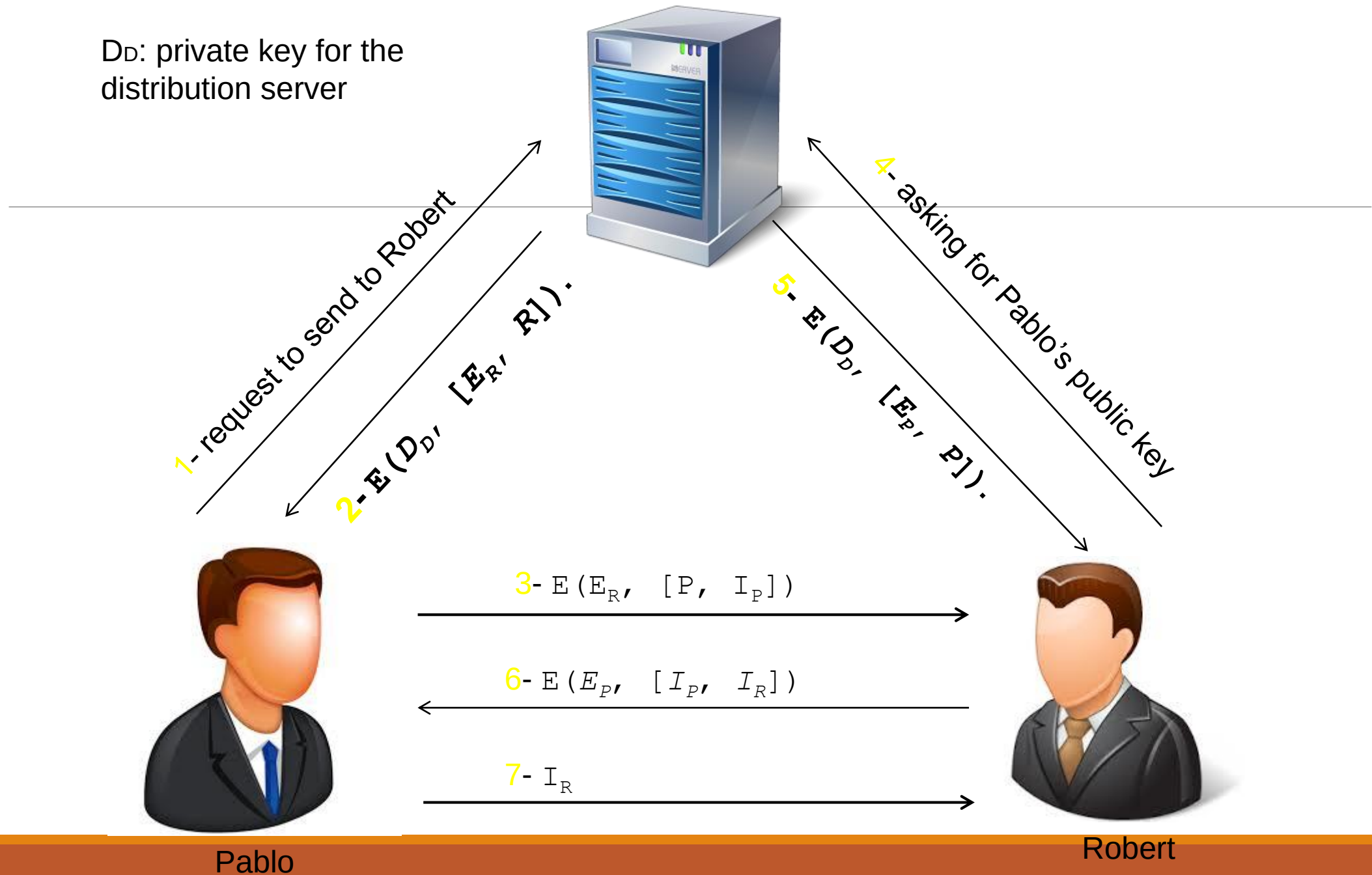
Robert sends a message to the server asking for Pablo's public key.

The server responds with $E(D_D, [E_P, P])$. Remember that D_D is the private key for the distribution server, E_P is the public key for Pablo and P is Pablo's identification.

Robert now sends $E(E_P, [I_P, I_R])$ to Pablo as proof that Robert has received the message.

Pablo now sends I_R back to Robert as proof that the reply to his message has arrived.

D_D : private key for the distribution server



4- Asymmetric Key Exchange Without a Server

Suppose that Pablo and Robert want to exchange a message, and each has a private and public key pair and each knows the public key of the other. Denote Pablo's public key as E_P and his private key as D_P . Denote Robert's public key as E_R and his private key as D_R .

Asymmetric encryption and decryption is very expensive in time so Pablo may want to use a symmetric algorithm to establish a shared key K_{PR} for symmetric algorithm S .

Pablo sends the message $E(E_R, E(D_P, K_{PR}))$.

- Only Robert can decrypt the outer layer using his private key D_R , so Robert is assured of secrecy.
- Robert can then use Pablo's public key E_P to decrypt the inner layer. Only Pablo could have applied on inner layer so Robert is assured of its authenticity.

Typically Robert and Pablo also exchange a test message to make sure that the other party properly received the message. For this Robert would send a random number called N and Pablo would return the $N+1$.

shared key K_{PR} for symmetric algorithm



1- $E(E_R, E(D_P, K_{PR}))$.

3- send a number N

4- reply $N+1$

2- decrypt using D_R
then E_P



9- Digital Signatures

A digital signature is a protocol that produces the same effect as a real signature: it is a mark that only the sender can make, but other people can easily recognize as belonging to the sender. Just like read signatures, a digital signature is used to confirm agreement to a message.

Digital signatures must meet two primary conditions:

- *Unforgeable:*
 - If person P signs a message M with a signature $S(P, M)$ it is impossible for anyone else to produce the pair $[M, S(P, M)]$.
- *Authentic:*
 - If a person R receives the pair $[M, S(P, M)]$ purportedly from P , R can check that the signature is really from P . Only P could have created this signature, and the signature is firmly attached to M .

Two additional requirements are also desirable:

- *Not alterable:*
 - After being transmitted, M cannot be changed by R or an *interceptor*.
- *Not Reusable:*
 - A previous message presented will be instantly detected by R .

Symmetric Key Digital Signatures

With private key encryption, the secrecy of the key guarantees the authenticity of the message as well as secrecy. If Sandy and the Bank have an encryption key in common, she can encrypt her request to transfer money. The bank can be sure of its authenticity because nobody else has Sandy's key.

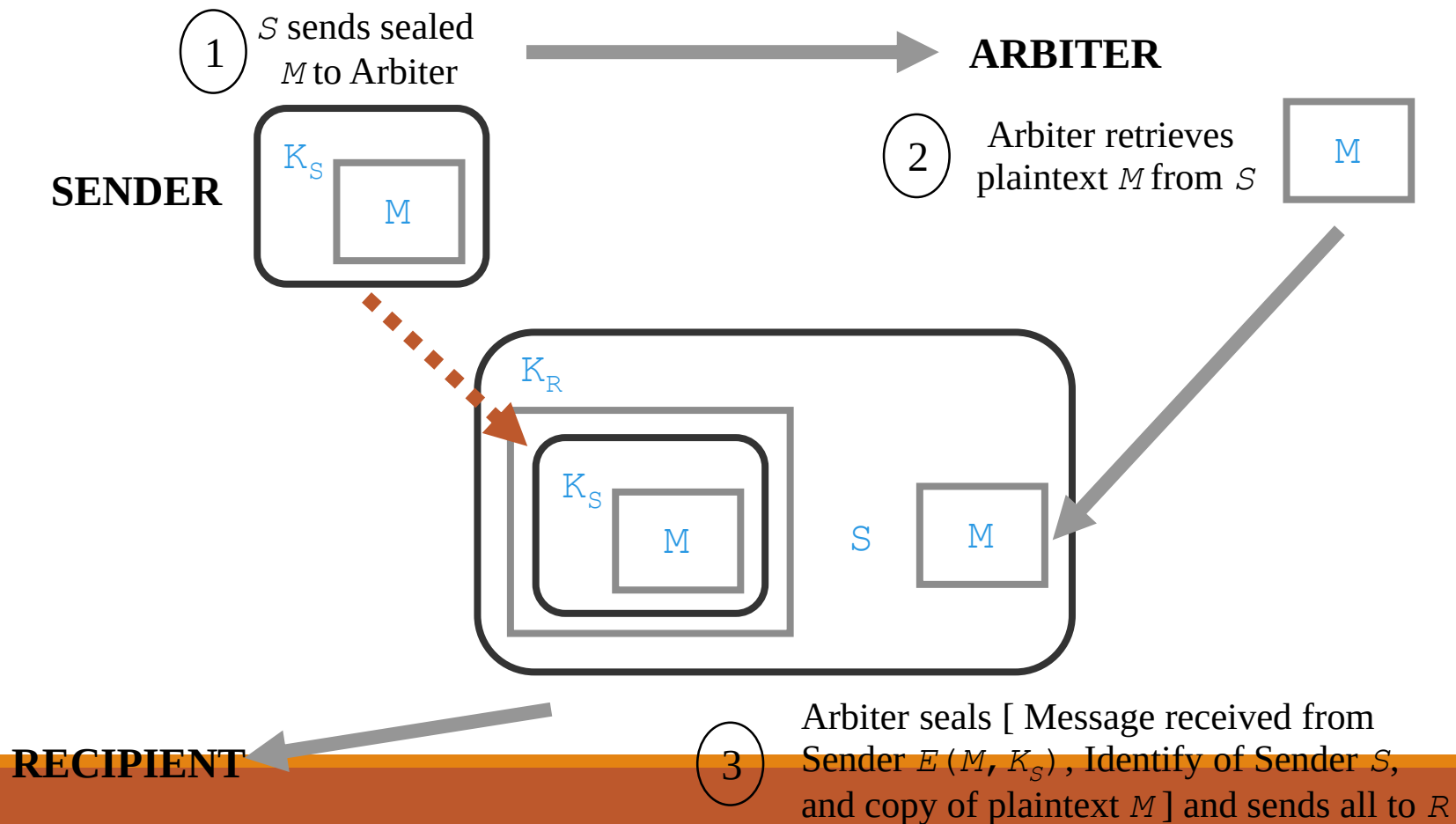
Conventional symmetric key encryption does not prevent forgery.

- Any one who knows the key can create a digital signature.
- Thus the bank has no protection against repudiation (denial of sending a message).

The repudiation problem can be solved if both the sender and the receiver use and arbitrator.

- Sender and arbiter share a key K_S
- Recipient and arbiter share a key K_R
- Identity of sender is S and identity of recipient is R
- Content of message between sender and recipient is M
- The arbitrator will use a sealing function. A sealing function is a mathematical function affected by every bit of its input. For example, the bytes of the input can be used as numbers and the sum of all input computed.

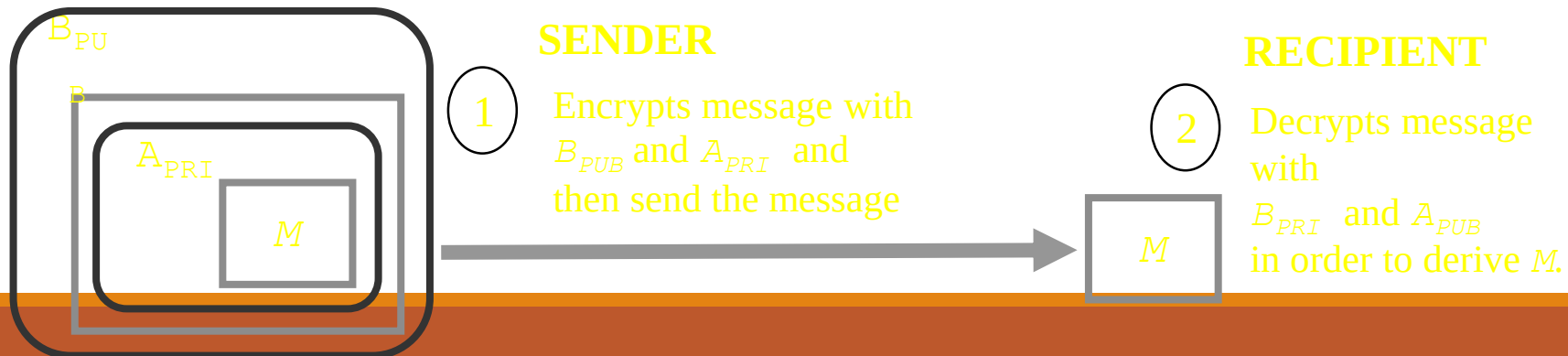
Symmetric Key Digital Signatures



Asymmetric Key Digital Signatures

The basic digital signature protocol using asymmetric encryption protocol is as follows.

- Alice encrypts a document M with her private key A_{PRI} , thereby signing the document, e.g. $E(A_{PRI}, M)$.
- Alice encrypts the message with Bob's public key B_{PUB} , and then sends the message C to Bob, e.g. $C = E(B_{PUB}, E(A_{PRI}, M))$
- Bob decrypts the message using his private key B_{PRI} and then Alice's public key A_{PUB} , thereby verifying the signature,
 - e.g. $M = D(B_{PRI}, D(A_{PUB}, M))$



Digital Signatures and Timestamps

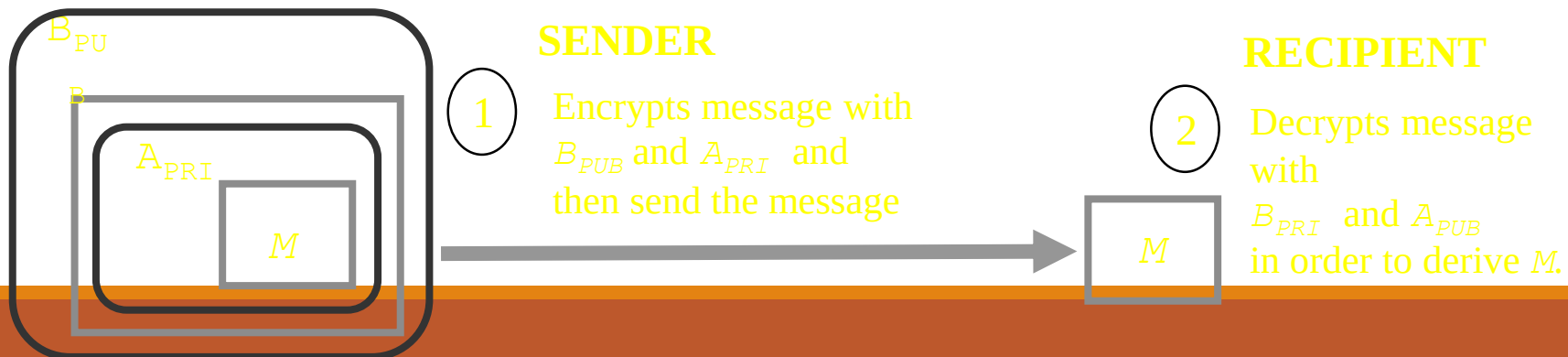
In order to stop a message that has been signed using a digital signature from being reused, we need to make use of timestamps.

The protocol is the same as the asymmetric protocol for signing documents except that the message contains a time stamp.

- e.g. $M = [\text{Message} , \text{Time-Stamp}]$

The timestamp will define when the message was sent.

- Two messages with the same contents and the same time-stamp will be ignored, thus a message can not be re-used.



Trusted Third Party (TTP)

- TTP provides a centralized authentication server to authenticate users to servers and servers to users.
- Relies on conventional encryption, making no use of public-key encryption

Distribution of Public Keys

can be considered as using one of:

- public announcement
- publicly available directory
- public-key authority
- public-key certificates

Public Announcement

users distribute public keys to recipients or broadcast to community at large

- eg. append PGP keys to email messages or post to news groups or email list

major weakness is forgery

- anyone can create a key claiming to be someone else and broadcast it
- until forgery is discovered can masquerade as claimed user

Publicly Available Directory

can obtain greater security by registering keys with a public directory

directory must be trusted with properties:

- contains {name,public-key} entries
- participants register securely with directory
- participants can replace key at any time
- directory is periodically published
- directory can be accessed electronically

still vulnerable to tampering or forgery

Public-Key Authority

improve security by tightening control over distribution of keys from directory

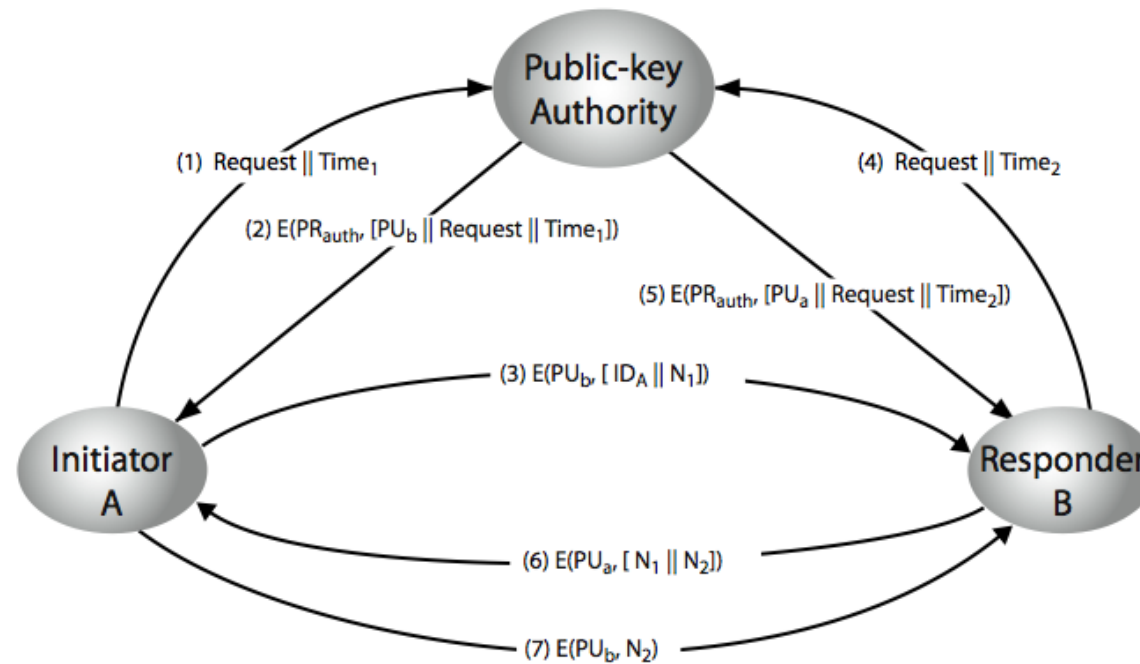
has properties of directory

and requires users to know public key for the directory

then users interact with directory to obtain any desired public key securely

- does require real-time access to directory when keys are needed

Public-Key Authority



Public-Key Certificates

certificates allow key exchange without real-time access to public-key authority

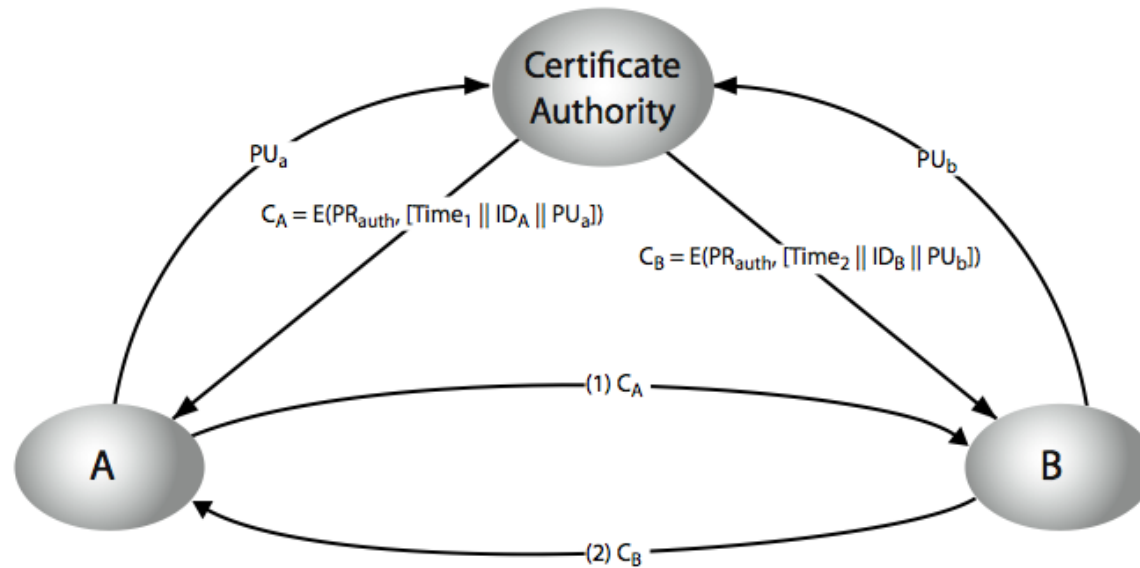
a certificate binds **identity** to **public key**

- usually with other info such as period of validity, rights of use etc

with all contents **signed** by a trusted Public-Key or Certificate Authority (CA)

can be verified by anyone who knows the public-key authorities public-key

Public-Key Certificates



Public-Key Distribution of Secret Keys

- use previous methods to obtain public-key
- can use for secrecy or authentication
- but public-key algorithms are slow
- so usually want to use private-key encryption to protect message contents
- hence need a session key
- have several alternatives for negotiating a suitable session

THANK YOU

